

Evaluating GPT-4o-mini for Unit Test Generation on Java and Python Functions of Medium Cyclomatic Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, Trần Xuân Lộc
Group 2 — SE1944, FPT University
Supervisor: L.T.Q.Chi

ABSTRACT

Unit-test generation with large language models (LLMs) is becoming increasingly common, but most existing studies still rely on simple, curated benchmarks that neither reflect the complexity of real-world code nor control for it. This study tests the one-shot capability of `gpt-4o-mini` on 120 real-world functions (60 Java, 60 Python) of medium cyclomatic complexity (5–10), mined from ten open-source repositories at pinned commits and evaluated directly inside their original projects. Its results are compared against EvoSuite and Randoop (Java) and Pynguin (Python) on branch coverage and mutation score, using non-parametric tests ($\alpha = 0.05$). Across all 120 functions, median coverage and median mutation score are both 0%. Only 28/120 suites (23.3%) compile and actually test the intended function; their median coverage is 75.0%, still below the 80% bar. Over suites with a measurable mutation score, the median is 36.84% for Python and 0% for Java, both under the 60% threshold. Against the baselines, `gpt-4o-mini` is beaten outright by both Java tools (EvoSuite and Randoop, $r = -1.00$ for each). It beats Pynguin only because Pynguin failed completely on this dataset (median coverage 0.0%). Within the CC 5–10 band there is no clear link between complexity and test quality; what decides the outcome is whether a suite compiles and tests the right function. Python failures are mostly bad imports, while Java calls the wrong method without any error — something only visible when running on real project code. Overall, one-shot LLM generation, without retrieval or repair, cannot yet replace automated test-generation tools on real code of medium complexity. The results support adding feedback loops and richer context when generating tests.

I. INTRODUCTION

At first glance, automated unit test generation using large language models seems nearly solved. For example, GPT-4o achieves 98.65% line coverage on TestEval [1], and GPT-4-class model panels report similar numbers on HumanEval-derived suites [2]. The functions behind those numbers, however, are short, self-contained, LeetCode-style problems. A test that performs well under these conditions has never had to deal with a real import graph, navigate a class hierarchy, or handle overloaded method signatures, and high line coverage alone does not guarantee that the test will actually catch faults.

When we instead sampled 120 functions from ten real projects — from Apache Commons, Gson and Joda-Time to Flask and Requests — we found that most of the test suites generated by `gpt-4o-mini` (92 out of 120, or 76.7%) never actually exercised the function they were supposed to test.

There are two gaps in current measurement practices that hide this problem. First, hardly any study controls for the structural complexity of the code under test: results are reported in aggregate, across functions with mixed and often unreported cyclomatic complexity (CC), so no one knows at what point LLM-generated tests start to fail (GAP-T). Because of this gap, we kept cyclomatic complexity fixed as a controlled variable, sampled only the 5–10 band, and measured the CC–quality correlation directly (RQ3). Second, branch coverage and mutation score are rarely combined in a single statistical test on a controlled sample (GAP-M), even though coverage alone can overstate quality — in our own baseline data, one `jsoup` function reaches 60% branch coverage while killing 0% of mutants (Section V).

Our design choices were shaped as much by lessons learned during implementation as by insights from the literature. The original proposal named Defects4J and CodeXGLUE as data sources; during preparation we found that CodeXGLUE is a code intelligence benchmark, not a corpus of executable functions, so we mined the dataset ourselves: 120 functions (60 Java, 60 Python) at pinned commits, all with a cyclomatic complexity between 5 and 10, as measured by Lizard. A pilot run then showed that functions extracted from their modules often cannot even resolve the names they reference — in fact, 10 out of our 12 Python pilot measurements were invalid for extraction-related reasons — so in the final experiment every generated test runs inside its original project. This measurement approach is important: measured this way, a score can no longer be reduced by artefacts from extraction, only by the quality of the generated tests themselves. We compare `gpt-4o-mini` (using one-shot prompting, temperature 0) against three automated baselines: EvoSuite and Randoop for Java (60 seconds per class), and Pynguin for Python (90 seconds per module), scoring branch coverage and mutation score, using non-parametric statistical tests with $\alpha = 0.05$.

We ask:

RQ1: Does GPT-4o-mini achieve at least 80% branch coverage on medium-complexity functions?

RQ2: Does GPT-4o-mini achieve at least 60% mutation score, and does it outperform automated baselines?

RQ3: Is there a negative correlation between cyclomatic complexity and test quality?

These two thresholds are not arbitrary: 80% is the branch coverage target most commonly quoted as the industry standard, while 60% is set intentionally above the roughly 34% mutation scores reported for LLM-generated suites in the studies reviewed in Section II; both were fixed in the approved proposal before any data were collected.

In summary, our contributions are: (i) a reproducible benchmark of 120 medium-complexity functions with full provenance; (ii) a measurement pipeline that scores tests *per function* inside the original projects and enforces a green-on-original check before any mutation analysis; (iii) an empirical comparison of GPT-4o-mini against search-based and random baselines on a complexity-matched sample; and (iv) a catalogue of the measurement pitfalls we encountered along the way — such as EvoSuite returning exit code 0 without generating a single test, or JaCoCo silently reporting 0% under EvoSuite’s instrumenting class loader — which would have corrupted our results if left unnoticed.

Section II reviews prior work; Section III details the dataset, generation, and measurement process; Section IV answers each research question; Section V interprets our findings, including two exploratory follow-ups; Section VI discusses threats to validity; Section VII summarises our work.

II. RELATED WORK

As we read the literature, three kinds of gaps kept showing up. Let us call them GAP-T, GAP-M, and GAP-D for brevity. The design of our work was mainly informed by the first: few works analyse GPT-4-family generation at a functional level, rarely do they study Python and Java together, and none of them identifies the point on the complexity axis from which performance begins to degrade — which we needed to know to design our experiment. GAP-M is less profound but also notable: coverage and fault-detection ability are seldom studied together in a controlled manner on a sample of tasks with varied complexity. And finally, GAP-D is found across much of the literature: without complexity restrictions, the datasets used tend to be small, artificial, or biased towards simple functions — the gap our own complexity-controlled dataset is built to close. The rest of this section is structured around these gaps.

A. LLM-based test generation on curated benchmarks

Many of these studies report very positive findings, mostly due to using curated benchmarks. Wang et al. [1] suggest TestEval — Python LeetCode-style tasks — and report up to 98.65% line and 97.16% branch coverage for GPT-4o. Kumar et al. [2] report 99.05% branch coverage for Gemini, GPT-4o, and Claude-3.5 on HumanEval-Java. Lira et al. [3] report a 90.69% success rate for Gemini 1.5-Pro on HumanEval when both the source code and docstring are provided. Chudic and Çalıkılı [4] and Jasti et al. [5] report even higher coverage

figures for similar tasks. These findings are hard to dismiss: the numbers are all very high.

However, these numbers come from narrow, self-contained tasks where the model writes isolated functions without having to navigate through a project, include or be included in other modules, or deal with any other real-world complexities. Additionally, these studies do not report or control for cyclomatic complexity, and therefore say little about how performance is affected on more complex functions, which is the main concern of this paper. Another issue is that the benchmark tasks are public: it is entirely possible that the model had seen similar tasks during training. The way we read these results is not “LLMs are great test generators”, but rather “LLMs are good at puzzles”.

In order to test this hypothesis, we created a benchmark of 120 functions from ten pinned open-source projects, limiting the scope to medium complexity ($5 \leq CC \leq 10$, as measured by Lizard) and excluding anything else. We did not attempt to optimise this range: it was specified in the design document for this project before any test generation had begun. The rationale behind this choice was that anything below 5 is too simple for any tool to handle effectively, and anything above 10 has not been studied in detail in the literature: our survey found no controlled experiments on the GPT-4-family with complexity restrictions. This left us with 5–10, the range that, according to the literature, had not been explored yet.

B. Degradation on real-world code

As soon as the model has to reason about real projects, the picture changes considerably. Haratian et al. [6] report that GPT-4o manages to compile tests in only 37% of Java pull requests, with 13% fail-to-pass, versus 36% for EvoSuite on the same set. Sapozhnikov et al. [7] report that over half of ChatGPT’s Java tests are malformed or non-compiling due to context-length issues. Zhang et al.’s TestBench [8] measures a 47.9–55.9% Java compile-error rate across nine open-source projects for CodeLlama, GPT-3.5, and GPT-4. Konstantinou et al. [9] evaluate a zero-shot pipeline (which includes `gpt-4o-mini` among the models) and find it produces many non-compiling or non-passing tests, with an aggregate 33.82% mutation score across six Java projects. For Python, Jain and Le Goues’s TestForge — an agentic test-generation pipeline based on GPT-4o — reaches 84.3% pass@1 on TestGenEval, but the same 33.8% mutation score [10].

Each of these works describes a different manifestation of the problem: malformed tests, compilation errors, unacceptable mutation scores. The trend, however, is obvious. These results are hard to compare due to varying criteria and, in some cases, different sets of target functions or repositories. When we attempted to bring them to a common scale while designing our experiment, almost all of them appeared reasonable in context, but none could be directly compared to the others. Overall, we find Haratian et al. [6]’s comparison of GPT-4o to EvoSuite particularly enlightening, as it shows that, on the same set of pull requests, GPT-4o significantly underperforms in both compilation and fail-to-pass rates.

Our results fall within the lower bound of this scale: 91.7% INVALID-or-no-touch for Java and 46.7% INVALID for Python. We define a suite as *effective* if it both compiles and executes at least one line of code for the target function. Suites that compile to the wrong method have no effectiveness. We encountered such cases often enough during development that they stopped being surprises. They often looked reasonable at first: a suite that compiled successfully, with tests whose names made sense. However, it became obvious during coverage and mutation testing that they did not actually test the target function.

C. Automated (non-LLM) baselines

The main non-LLM baselines are represented by EvoSuite [11], Randoop [12], and Pyguint [13]. EvoSuite uses genetic algorithms and is the default search-based test-generation tool for Defects4J [14], [15], [16]. Randoop uses feedback-directed random testing, and Pyguint implements the DY-NAMOSA algorithm for Python. None of these tools makes use of natural-language prompts, which is why none of them report compilation failures.

When evaluating these tools, one of the first issues we had to address was the time budget. With default settings, EvoSuite’s results were considerably better than Randoop’s. The discrepancy was significant enough that we suspected a mistake at first. It turned out that EvoSuite’s default budget was 60 seconds per class, while Randoop’s was 10 seconds per class. We adjusted the budget to 60 seconds per class for both tools and 90 seconds per module for Pyguint. We also shifted the emphasis from per-call metrics to per-class (Java) or per-module (Python) ones. As a result, we were able to show that, with a comparable time budget, EvoSuite significantly outperformed Randoop: 55.0 median branch coverage versus 21.1. Had we not made this adjustment, our conclusions would have been entirely different.

D. Closing the one-shot gap: repair loops, retrieval, and context

When the initial prompting approach proved insufficient, most works tried adding additional layers on top of the model. Repair loops reason about compiler errors and test failures and attempt to fix them by feeding this information back into the model, with work such as Logic-CoT [17], template-based repair [15], and YATE [18] representing this direction. DISTINCT [19] takes a different approach by steering regeneration with description-guided consistency analysis rather than directly analysing compiler output. Retrieval-augmented methods attempt to provide real API documentation and class skeletons based on call-graph context rather than raw source code [20]. Best-of- N sampling replaces the single sample with a distribution and filters out any candidates that fail to compile, while TestDecision pushes decision-making to the model itself, using greedy optimisation and reinforcement learning to direct test-suite construction [21].

Each of these approaches has seen varying degrees of success. The central problem they present is that it becomes

very difficult to reason about the contribution of individual parts: how much is due to the model’s own capabilities, and how much to the added layers? We were particularly interested in exploring the capabilities of one-shot prompting without additional scaffolding. It is arguably the most practical setting of all: a single model, a single prompt, a single sample. While most works in this space use more involved pipelines, they usually do not publish enough detail on the underlying distribution to compare their results to a one-shot baseline. This baseline is what we chose, and it dictated much of our design. If we knew how prompting-based methods performed on medium-complexity code, we could then attempt to understand how much better the enhanced approaches really are.

E. Coverage vs. mutation score as a validity concern

JA-008 is the function that taught us about validity. With 60% branch coverage and 0% mutation score, it took us some time to realise that there was nothing wrong with our setup. The baseline suite was executing the function under test, but it failed to assert any of the outputs — there were no meaningful assertions in any of the tests the suite generated. Moradi Dakhel et al. [22] and Antal et al. [23] describe a similar pattern in their work: in each case the test suite had coverage, but it failed basic mutation analysis. The most prominent example we are aware of is TestForge’s 84.3% pass@1 combined with a 33.8% mutation score [10], which represents an agentic approach with extensive feedback built into it. JA-008 appears similar to these cases: a test suite that touches the code without actually testing it.

Most works report either coverage or mutation score, occasionally comparing the two on the same set of functions, but rarely drawing actual conclusions from the juxtaposition. Additionally, mutation score frequently gets reported alongside non-standard testing infrastructure [24]. We attempt to mitigate these issues by reporting threshold-test results alongside detailed mutation analysis, using standard testing libraries, for the same set of function identifiers as used in RQ1.

III. METHODOLOGY

A. Dataset

The proposal mentioned Defects4J and CodeXGLUE as sources. We excluded the latter: CodeXGLUE is a benchmark of code-intelligence tasks, including summarisation and translation, not a set of functions amenable to coverage or mutation analyses; nor did any of the papers surveyed for this proposal actually evaluate these tasks on CodeXGLUE alongside any of these measurements. We document this exclusion as amendment v1.2 (finalised by the team on 2026-07-02, hence before the experiment) and record functions from our own ten repositories at particular commits. Eight are Java projects overlapping with Defects4J’s subjects in their target programs: `commons-cli`, `commons-math`, `commons-csv`, `commons-collections`, `gson`, `jsoup`, `joda-time`, and `jfreechart`; two are Python projects, `flask` and `requests`. URLs, licences, commit hashes,

and dates of download are recorded for every repository to facilitate reproduction; the final sample contains 120 functions — 60 in each language — with cyclomatic complexity between 5 and 10 according to Lizard. Of these, 48 Python functions come from a single project, `flask`, rather than being split evenly between two as the porting procedures suggested; we retained this skew (resampling the dataset after observing the result would itself have been a worse methodological choice) and discuss it at length in Section VI.

B. Test generation

LLM. All 120 test suites were generated by a single call to `gpt-4o-mini-2024-07-18` (i.e., the dated snapshot; hence a rerun of this experiment would use the same weights), `temperature=0`, `top_p=1`, `max_tokens=2048`, one-shot, with a worked example preceding the target function in each prompt. The proposal erroneously mentioned `gpt-4o`; the team’s API key had access to `gpt-4o-mini` only at the moment the experiment was run, and we document this change as amendment v1.1 (documented 2026-06-28 and finalised by the team 2026-07-02, hence also before the experiment). This model has been evaluated on the same task in this literature before [9], which keeps us within the evidential basis of this report. None of the research questions, metrics, thresholds, or statistical tests were affected by either amendment.

Baselines. EvoSuite [11] and Randoop [12] generate the Java comparison suites; 60 seconds are allocated to each class under test for test generation. That allocation was not fixed initially: the pilot experiment spent 60s on EvoSuite and 10s on Randoop (a bias we only became aware of when comparing the results), while the final run equalised the budgets (see Section VI). Pynguin [13] is used for Python with 90s per module under DYNAMOSA. Hypothesis was a viable baseline for the initial comparison but was not actually used for the reported results.

C. Measurement

We declined to measure functions in isolation; all test suites are executed in the target project at the commit corresponding to the function being tested (recorded in the dataset’s provenance record; see the URLs, licences, hashes, and dates in the Dataset subsection), with whatever environment and dependencies were present there at the time. The reason for this design choice was that many of our sample functions only manifested their documented behaviour while resolving their imported names inside the module or package they belonged to (see the pilot result leading to this insight in Section VI).

Java. Tests are compiled and run with Maven 3.9.16; JaCoCo 0.8.12 measures branch coverage while PIT 1.17.2 [25] performs mutation analysis. Both report coverage and mutation scores at the class level, while our unit of analysis is the function; hence the values reported for each function are the intersection of the reported lines with the `[start_line, end_line]` of the target function across all reports. The mutation score is the fraction of PIT mutants reported within that line range to be killed by the test suite. There were two

bugs in this pipeline affecting our results: the PIT timeout was not handled by the measurement script, causing one entire run to be dropped; and `jacoco.xml` and `mutations.xml` reports from the previous function erroneously carried over into the next run in the same module. The latter bug would have caused a function to erroneously report coverage of lines belonging to another function (and hence artificially inflate its score), so the final script clears the report directory between runs. Finally, GPT test suites had a much higher fraction of compilation errors than the baseline tools’; hence we compiled and measured each GPT test suite individually, per function, per run, rather than in bulk. This avoids an entire test suite in a batch failing to compile because another test suite in the same file has syntax errors.

Python. Functions are measured in an editable install of the pinned `flask` (3.2.0.dev, pinned 2026-05-31) and `requests` (2.34.2, pinned 2026-06-15) checkouts; `coverage.py` 7.15.1 measures branch coverage, cut to the function’s line range as in Java. Mutation analysis is performed with two different instruments, one for each test-suite flavour. The `gpt-4o-mini` suites are scored with `mutmut` 2.4.4, with all lines outside the target function’s range marked as `# pragma: no mutate`. Only test cases passing on the unmutated source are considered; the score is the fraction of mutants with a non-surviving status (killed or timed out) over all mutants generated in the line range. The Pynguin suites are scored with a home-grown script using Python’s standard-library `ast`: it replaces `+` and `-`, `>` and `>=`, `<` and `<=`, `==`, `and`, and `or` with their opposites, negates boolean literals, increments numeric literals by 1, and applies any of these changes to exactly one site in the target function’s line range, up to 20 mutants per function. Every such mutant is scored by rerunning the whole test suite with a 90s timeout: the suite killed the mutant if it failed or timed out; the source code is reverted to its original state after each run. Functions with no mutable site in range are reported as missing rather than scored 0, since no mutants could be generated for them. Neither script attempts to detect equivalent mutants; hence both report scores as a fraction of all mutants generated in range, for the most conservative estimate. Both instruments use the same green-on-original gate and line-range scoping; hence the only difference affecting the comparison is the set of operators replaced. This makes the RQ2-B comparison between `gpt-4o-mini` and Pynguin unsuitable for directly assessing the equivalence of their mutation scores; we discuss this as a construct-validity threat in Section VI. The Java RQ2-B comparison is unaffected: PIT scores both test-suite flavours with the same instrument. Limiting mutation analysis to the target-function line range is itself another conservative choice: by avoiding helper functions and static variables, it prevents incidental coverage and mutation of irrelevant code, maximising the score difference between test-suite flavours for each function, at the expense of potentially missing faults in wider contexts. Neither step involved any randomness, given the pinned checkout and the model snapshot at `temperature=0`.

Any suite failing to compile, import, or run is marked `INVALID`, with both metrics scored at 0% (see §5.1 of the proposal). Before attempting to run the mutation analysis, each test suite must pass on the original, unmutated source code. Mutation analysis run on a failing test suite would artificially inflate the score: Section VI describes the pilot bug that led to this check. An `INVALID` suite — one that failed to compile or run, or failed on the original source — is not the same as a suite that compiled and ran but failed to cover any of the target lines (i.e., `compiled=1`, `coverage = 0`): both receive 0% coverage, but for different reasons, and both are tracked as separate outcome types in Section IV for easier downstream analysis.

D. Statistical analysis

All tests are performed at $\alpha = 0.05$; all are non-parametric. Coverage and mutation scores are percentages bounded between 0 and 100, so normality was never assumed for these variables. RQ1 requires one test: a one-sample Wilcoxon signed-rank test of branch coverage against the 80% threshold. RQ2 requires two: a one-sample Wilcoxon signed-rank test of mutation scores against the 60% threshold, and a paired Wilcoxon test (matched by `function_id`) of the same against each baseline, with matched-pairs rank-biserial r as the effect size. Finally, RQ3 requires a Spearman correlation of CC with each quality metric. Every statistic is reported at two levels: *all* ($N = 60$ per language, with `INVALID` or non-touching test suites scored at 0% per the pre-registered procedure) and *effective* (test suites that successfully compiled and touched the target lines). The two levels are never reported separately without the other, since omitting the all-level would artificially inflate the statistics: an effective suite always has higher coverage than an invalid one (0% versus some positive number), so the effective figure is always higher than the all figure by construction. Section V contains more context on this point.

IV. RESULTS

We evaluate one-shot `gpt-4o-mini-2024-07-18` unit tests on 120 functions (60 Java from 8 Defects4J subject programs; 60 Python from `flask/requests`) at cyclomatic complexity 5–10, with branch coverage measured by JaCoCo (Java) / `coverage.py` (Python) and mutation score by PIT (Java) / `mutmut` for the LLM suites and a custom AST-level engine for the Pinguin baseline (Python; Section III). All tests run at $\alpha = 0.05$, with matched-pairs rank-biserial r (Wilcoxon) and Spearman ρ (RQ3) as effect sizes. Per the pre-registered scoring rule, any suite that fails to compile or fails to execute scores 0% and is marked `INVALID`. Every metric below is reported at two levels side by side — *all* ($N = 60$, `invalid = 0`) and *effective* (only the valid suites that actually exercise the target) — since the effective figures are conditional and should never be read without the all-level figures next to them.

A. Test validity (a primary finding)

On the Python side, only **32/60 (53.3%)** of the generated suites executed even one test; the remaining **28/60 (46.7%)**

were `INVALID`. Of the 32 valid suites, just **23** actually exercised the target function (branch coverage > 0); the other 9 ran without ever touching the target code (mock or self-implementation). Breaking the 28 invalid suites down further:

- **23 — ImportError** (LLM-attributable): the test imports a symbol that does not exist in the target module (e.g. importing an instance method as a module-level function). A small subset of these are dataset/platform artefacts rather than LLM faults (e.g. `proxy_bypass_registry` is Windows-only; `merge_environment_settings` is a method, not module-level).
- **5 — TypeError/AttributeError** (environment-sensitive): these may vary with the exact `flask/requests` dependency versions and should be confirmed in the canonical pinned environment.

That 46.7% invalidity rate is the first main finding, and the breakdown matters: 23 of the 28 failures are LLM-attributable import errors, not environment noise. The recurring shape — importing an instance method as if it were a module-level function — is a misreading of the API’s namespace, not a random slip.

Java tells a different story on the surface: **51/60 (85.0%)** of the generated suites compiled and ran, with only **9/60 (15.0%)** `INVALID` due to outright compile failure. That lower Java invalidity rate is misleading, though. Of the 51 compiling suites, only **5** actually exercised the target method (branch coverage > 0); the other **46** compiled, ran, and touched *none* of the target’s lines. A closer look at a sample of these (JA-002, JA-020) reveals the dominant pattern: *wrong-target invocation*. The LLM writes a call that is syntactically valid and resolves to a real, accessible method — just not the one it was asked to test. For instance, `CommandLine.getOptionValues(...)` gets called when the sampled function is actually the unrelated, nested `CommandLine.Builder.getOptionValues(...)`; elsewhere, a same-named method inherited from higher up the class hierarchy gets invoked instead. This is a fundamentally different failure mode from Python’s `ImportErrors`. Here, the Java compiler’s own overload/name resolution silently accepts a plausible substitute, so the suite is counted as “compiling.” At the same time, it measures nothing about the function it was meant to test. Once both effects are combined, only **5/60 (8.3%)** of Java suites end up effective, compared to 23/60 (38.3%) for Python. Among the 9 truly `INVALID` suites (non-compiling), 6 target `AccurateMath` — a numeric utility class packed with heavily overloaded, edge-case-dense methods — and 3 target `Gson`’s generically-typed adapter API (`getAdapter`, `getDelegateAdapter`, `fromJson`); just 1 of the 9 (itself an `AccurateMath` function) involves a *private* target method. The failures, in other words, cluster around *API surface* — heavy overloading, generics — far more than around method visibility. That distinction matters for anyone designing a fix: making the target `public` could have mattered in at most 1 of the 9 cases, while a class skeleton

— which method exists, what its signature looks like, how it differs from same-named neighbours — addresses the pattern we actually observe. Section V-F tests exactly that.

B. RQ1 — Branch coverage vs. 80%

Across all 60 Python functions the median branch coverage is **0%**: for at least half the sample, the generated suite covered no branch of its target at all. Only 23 of the 60 suites exercised any part of the intended code. The effective subset looks better — median **75.0%** ($n = 23$; 11 of the 23 reach 80%, and 6 hit full coverage) — but still sits under the 80% bar, and the one-sample Wilcoxon finds no support for the threshold ($p = 0.67$, $r = -0.10$). Java repeats the pattern one level lower: all-function median **0%**, effective median **50.0%** on a much smaller subset ($n = 5$, the 8.3% effective rate; $p = 0.91$, $r = -0.60$). Pooling both languages changes nothing: $n = 120$ median **0%**; pooled effective median **75.0%** ($n = 28$, $p = 0.84$, $r = -0.22$). Whichever language or aggregation we pick, GPT-4o-mini stays short of 80% branch coverage (Fig. 2). The effective-subset numbers are conditioned on the model having already succeeded at the hard part — compiling and hitting the right target; the all-level median of 0% is the honest number.

C. RQ2 — Mutation score

(A) vs. 60%. The median mutation score across all 60 functions is **0%**, with a measurable score produced for only 17/60. Restricting to that measured subset — green, passing tests that actually touch the code — the median rises to **36.84%** ($n = 17$; just 4/17 reach $\geq 60\%$), well below the threshold (one-sample Wilcoxon, $p = 0.99$, $r = -0.66$). Java’s all-function median is also **0%** ($n = 60$); even when restricting to the 54 functions where PIT produced a measurable score, the median remains at **0%** (one-sample Wilcoxon, $p = 1.00$, $r = -1.00$) — consistent with the 8.3% effective rate: nearly all Java-generated suites fail to kill any mutants. Pooled across both languages (measured subset, $n = 71$), the median is likewise **0%** ($p = 1.00$, $r = -0.97$). H_0 is not rejected in either language: the tests generated are weak at killing mutants, with Java even weaker than Python — a 0% median even among the 54 functions where PIT generated mutants means those suites execute code without ever placing a bet on what the output should be.

(B) vs. baseline. Paired Wilcoxon tests — gpt-4o-mini against each baseline, matched by `function_id` on mutation score — tell a consistent story. gpt-4o-mini *loses substantially* to both automated Java baselines: against EvoSuite ($n = 54$, $p < 0.001$, $r = -1.00$) and against Randoop ($n = 54$, $p = 0.003$, $r = -1.00$), an effect size at its theoretical maximum, meaning EvoSuite and Randoop win *every* pair where the two methods differ at all (Fig. 1). The direction flips against Pyguint (the Python baseline): gpt-4o-mini *considers* outperforms it ($n = 13$ paired functions, difference in medians +36.84 pp — GPT median 36.84 vs. Pyguint 0.0 — $p = 0.010$, $r = +0.85$). However, this apparent advantage does not indicate that gpt-4o-mini produces

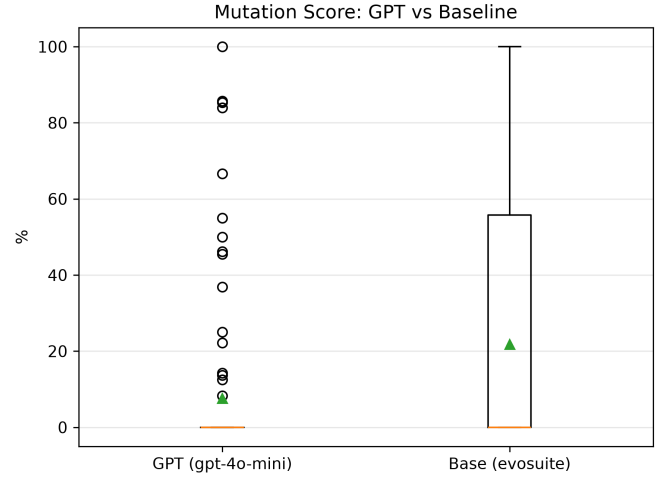


Fig. 1. Mutation score, gpt-4o-mini vs. EvoSuite, matched by function ($n = 54$ paired Java functions). EvoSuite wins in every pair where the two differ ($r = -1.00$, $p < 0.001$).

effective tests, but rather illustrates a practical shortcoming: Pyguint’s own median branch coverage on this dataset is **0.0%** (Section VI). Neither tool, in other words, reliably produces mutation-killing tests for real code in this band; gpt-4o-mini’s nominal win exists only because the baseline collapsed. Under Python 3.10/DYNAMOSA with a 90-second budget per module, Pyguint itself repeatedly times out, hits a `dill pickling error` on modules importing `_json`, or exits via an internal tracer abort, exporting zero test cases for most modules. H_0 is therefore rejected twice, in opposite directions — and the two rejections mean opposite things. The Java result reflects a genuine quality gap: EvoSuite and Randoop systematically produce tests that kill mutants, and gpt-4o-mini does not. The Python result reflects a tooling failure. Reading both as “the model beats some baselines but not others” would be the wrong takeaway.

D. RQ3 — Complexity vs. quality

For Python’s effective subset, Spearman $\rho = +0.18$ ($p = 0.41$, $n = 23$) shows *no* significant negative correlation between cyclomatic complexity and branch coverage inside the CC 5–10 band. Java’s effective subset ($n = 5$) yields $\rho = +0.13$ ($p = 0.83$); again, no significant correlation, though the sample is tiny. Pooled ($n = 28$), $\rho = +0.16$ ($p = 0.40$). H_0 is not rejected in either language: inside the narrow CC 5–10 band we sampled, complexity does not predict coverage quality among the functions that produce any signal at all — what dominates instead is the binary compiles/touches-target outcome discussed above, not a graded decline as CC rises. The pre-registered decision rule for RQ3 was $\rho < -0.5$ (at $p < 0.05$); the observed correlations fail it by sign alone — if anything, positive values are the wrong sign for the complexity-hurts-quality hypothesis, though none is distinguishable from zero.

TABLE I

PER-RQ SUMMARY (GPT-4O-MINI). “ALL” = 60/LANG. (INVALID/NO-TOUCH = 0); “EFF.” = EFFECTIVE SUBSET (COMPILED & TOUCHES TARGET); “MEAS.” = SUBSET WITH A MEASURABLE MUTATION SCORE.

RQ	Metric	Level	Median	p	Effect
RQ1 Py	Coverage $\geq$ 80	all (60)	0%	—	—
		eff (23)	75.0%	0.67	$r=-0.10$
RQ1 Java	Coverage $\geq$ 80	all (60)	0%	—	—
		eff (5)	50.0%	0.91	$r=-0.60$
RQ1 All	Coverage $\geq$ 80	all (120)	0%	—	—
		eff (28)	75.0%	0.84	$r=-0.22$
RQ2A Py	Mutation $\geq$ 60	all (60)	0%	—	—
		meas. (17)	36.84%	0.99	$r=-0.66$
RQ2A Java	Mutation $\geq$ 60	all (60)	0%	—	—
		meas. (54)	0%	1.00	$r=-1.00$
RQ2B Java	GPT vs EvoSuite	paired (54)	—	<0.001	$r=-1.00$
RQ2B Java	GPT vs Randoop	paired (54)	—	0.003	$r=-1.00$
RQ2B Py	GPT vs Pynguin	paired (13)	$\Delta_{\text{med}} +36.84\text{pp}$	0.010	$r=+0.85$
RQ3 Py	CC $\sim$ Cov	eff (23)	$\rho=+0.18$	0.41	+0.18
RQ3 Java	CC $\sim$ Cov	eff (5)	$\rho=+0.13$	0.83	+0.13
RQ3 All	CC $\sim$ Cov	eff (28)	$\rho=+0.16$	0.40	+0.16

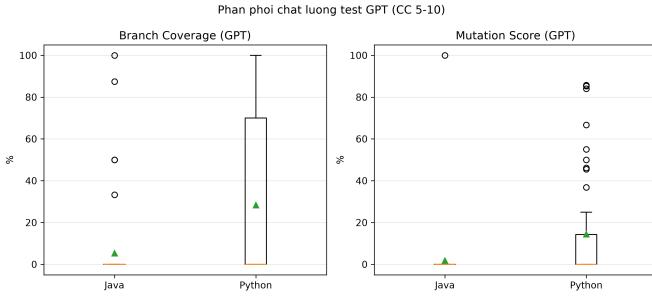


Fig. 2. Branch coverage and mutation score of gpt-4o-mini on the effective subset (dashed: 80% / 60% thresholds). The all-function medians are 0% (see text).

E. Summary of findings

Neither the 80% coverage nor the 60% mutation threshold is met in either language ($N = 120$). Even in the most favourable group, the effective subset, coverage is 75.0%, still below 80%. When counting all 120 functions, the median is 0%. There is no clear link between complexity and quality within CC 5–10 for either language. What matters most here is test validity, and it breaks differently in each language: Python mostly dies at compile time (46.7% INVALID, dominated by `ImportError`), while Java mostly compiles and then measures nothing (76.7% compile yet touch 0% of the target), through silent wrong-target/overload resolution. Effective suites make up only 8.3% of the Java sample and 38.3% of the Python one. On paired comparisons, gpt-4o-mini loses decisively to both automated Java baselines (EvoSuite, Randoop; $r = -1.00$ each) but beats the Python baseline Pynguin, whose own coverage on this dataset sits at 0.0% because of tool–environment issues rather than any real task difficulty (Table I).

F. Threats to validity

Conditional subset. The effective-level medians (75%/50%/37%/0%) are conditioned on suites that both

compiled *and* touched the target, so they *overstate* raw ability relative to the all-level (0%) — which is why we report both at every RQ rather than either alone. **Environment.** The Python figures come from a clean editable-install environment; the exact invalid rate is sensitive to the precise `flask/requests` dependency versions in use. The Python baseline figures needed a separate Python 3.10 virtual environment just for Pynguin, since Pynguin’s bytecode instrumentation silently produced empty test suites under Python 3.14, the platform otherwise used for GPT/measurement scripting; this is documented in Section VI as an environment amendment, not as a change to any pre-registered metric or threshold. **Sample size.** Effective n is small in every cell (5–28), and particularly so for Java RQ1/RQ3 ($n = 5$) — those point estimates are indicative, not precise. **Single sample.** With one generation per function (one-shot, $\text{temp} = 0$), we have no per-function variance estimate. **Cross-language tooling.** The Python harnesses (`coverage.py`, `mutmut`, the AST mutation engine) and the Java ones (JaCoCo, PIT) differ enough in implementation that cross-language comparisons in this section stay descriptive — we never test one language’s scores against the other’s. **Baseline-tool confound.** The RQ2B win against Pynguin reflects Pynguin’s own tooling trouble on this environment (Section VI), not evidence that gpt-4o-mini produces strong tests in absolute terms.

V. DISCUSSION

A. Coverage is not test quality

Branch coverage and mutation score pull apart in our own data, on a controlled sample — the very pattern that motivated this study’s paired metrics. JA-008 is the clearest case: a baseline suite reaches 60% branch coverage while killing 0% of mutants. The suite runs the function, visits its branches, and exits without ever asserting anything that would distinguish a correct return value from a mutated one. It is not a broken test — it just has no opinion about the code. The aggregate RQ1/RQ2 numbers split the same way. Effective-subset coverage (75.0% Python, 50.0% Java) reads as far

less alarming than the mutation numbers over the measurable subsets (36.84% Python, 0% Java). Someone shown only branch coverage would call GPT-4o-mini’s tests moderately competent; someone shown only mutation score would call them close to worthless. Both descriptions are accurate; the metric you choose determines the conclusion you reach, which is exactly why we report both, at both the *all* and *effective* levels.

B. Where does the “breaking point” actually sit?

GAP-T asked for the cyclomatic-complexity threshold at which LLM test generation starts to degrade. RQ3 finds no significant CC–quality correlation within CC 5–10 in either language ($\rho \in [+0.13, +0.18]$, all $p > 0.4$), and next to RQ1/RQ2 that result argues against the “breaking point” framing itself: quality does not decline *gradually* across CC 5–10 at all. What dominates is a *binary* outcome, decided almost independently of CC within this band — whether the generated suite compiles, and whether it invokes the sampled method rather than a same-named neighbour or an inherited overload. A CC 5 function and a CC 10 function in our sample are roughly equally likely to fail that gate. Complexity may still matter to LLM test generation in general; inside the narrow, pre-selected medium-complexity band we sampled (chosen to stay clear of the extremes), something else drives most of the variance. Pinning down a true breaking point would require sampling low-CC (< 5) and high-CC (> 10) functions for contrast, which we leave as future work (Section VII).

C. Cost is not the primary constraint; correctness is

Generating all 120 one-shot suites cost well under a dollar in API charges (the pre-registered budget estimated \$1; amendment v1.1, §8.2) and took only a few minutes. EvoSuite and Randoop each ran a 60-second search budget per class, so covering the same 60 Java functions took tens of minutes before any measurement began. On price and speed, the LLM (gpt-4o-mini-2024-07-18) wins outright. On exercising and killing mutants on the intended target, it loses to both automated Java baselines with an effect size of $r = -1.00$ — every non-tied pair favours the baseline. Cheap and fast, then, but not reliable, and the gap is wide. Spending part of the saved time on a few compile-and-retry attempts (a feedback loop) could narrow it without approaching EvoSuite or Randoop’s total runtime.

D. The INVALID rate as a finding, not a bug

We did not filter out functions that are structurally hard for a one-shot external test to reach — `private` target methods being the obvious example, at 17/60 of the Java sample. Removing them after seeing Java’s low effective rate would have meant selecting the sample on the very outcome under measurement, against the pre-registered protocol (amendments v1.1/v1.2, §VI). The data then showed that such a filter would have cut the wrong variable anyway: visibility is *not* the dominant cause. A `private` target is involved in only 1 of the 9 truly non-compiling Java functions and only 14 of

the 46 that compiled but touched nothing (Section IV). The dominant failure mode — wrong-target invocation via silent overload/name resolution — would have gone unexplained; the outright compile failures cluster separately, around the API-heavy `AccurateMath` and `Gson` classes (Section IV).

Nor is the pattern unique to our sample. Haratian et al. [6] report a 63% compilation-error rate for GPT-4o on 353 real Java pull requests — roughly two-thirds of our Java INVALID-or-no-touch rate (91.7%), and the same qualitative story: one-shot, zero-context generation on real, uncurated code compiles far less reliably than benchmark-style evaluations suggest. Sapozhnikov et al. [7] report over 50% malformed or non-compiling ChatGPT-generated Java tests, and Kumar et al. [2] trace weak single-pass results directly to the *absence of an iterative-repair loop*. The fixes the literature reaches for — feeding compiler or test-failure output back to the model (Zeng et al. [17]; Xue et al. [19]), retrieving real API signatures instead of raw code (which targets exactly the wrong-target-invocation pattern we observe), or sampling several candidates and keeping whichever compiles — all step outside the strict one-shot setting evaluated here. Our numbers are therefore a lower bound on what GPT-4o-mini can do with this codebase, and a direct demonstration of why the field has moved toward feedback-augmented and retrieval-augmented pipelines rather than static one-shot prompting.

E. Why gpt-4o-mini “beats” Pynguin

The RQ2-B result against Pynguin ($r = +0.85$, favouring gpt-4o-mini) says more about the baseline than about the treatment. Pynguin’s own median branch coverage on this dataset is 0.0% (Section VI), driven by search timeouts and a `dill` serialisation failure on modules that import `_json` — neither of which touches gpt-4o-mini’s ability to write tests. We report the comparison anyway: it is the pre-registered baseline for Python (amendment v1.2), and quietly dropping an inconvenient result would be its own validity threat. It is not evidence that gpt-4o-mini writes good Python tests in absolute terms — RQ1/RQ2-A already rule that out.

F. RQ4 (exploratory, post-hoc): does real API context help?

Wrong-target invocation, the dominant Java failure above, motivated a small *exploratory, post-hoc* follow-up: the same 120 functions, the same model/temperature/exemplar, but with the target class’s (Java) or module’s (Python) real public-API skeleton — method signatures, nested-class structure, the distinction between a module-level function and a class method — extracted from the pinned source with `javalang/ast` and inserted into the one-shot prompt. The idea came only after the $N = 120$ results were in, so it sits outside the pre-registered RQ1–RQ3 and is reported separately; folding it into the confirmatory analysis would be HARKing. The new suites went into a separate directory and results file, leaving the registered $N = 120$ data untouched.

Java. The compile rate does not move (51/60 in both arms), but the *effective* rate — suites that actually exercise the target — more than doubles: 11/60 (18.3%) with context

versus 5/60 (8.3%) without. Paired by function, 7/60 improve, 1/60 regresses, and 52/60 stay unchanged; a one-sided paired Wilcoxon test falls just short of significance at $\alpha = 0.05$, though 7 favourable pairs against 1 unfavourable is suggestive on its own, and the effect size among the non-tied pairs is moderate-to-large. Mutation score does not improve at all ($n = 48$ pairs; both medians 0%). Context, then, fixes the structural gap — which method to call — and leaves the behavioural gap untouched: the model still does not assert what the method should return, only that it can be called. The two problems need different interventions. Wrong-target invocation is an API-navigation failure; weak assertions are a test-design failure. A class skeleton addresses the first; the second likely needs examples of what a meaningful assertion looks like for the specific function under test.

Python. Judged by the pre-registered whole-suite-must-pass criterion, context looks *worse* (7/60 compiled versus 32/60) — a measurement artefact, not evidence of harm. Counted at the level of individual test functions rather than whole files, the two arms are near-identical: 267 test cases at 33.3% passing with context, versus 233 test cases at 32.2% passing without. What context actually does is push gpt-4o-mini to write *more* test cases per file — plausibly because it now sees more of the API surface — and since our Python INVALID criterion demands that every test in a file pass, more tests per file mechanically raise the odds that at least one of them trips over an unrelated behavioural bug. We traced one such case, `flask.logging.has_level_handler`: an identical misunderstanding of logger-hierarchy propagation appears in *both* arms, unrelated to context at all. That leaves a second-order validity threat for future work: an all-or-nothing pass criterion penalises test-suite thoroughness the moment any single assertion is wrong, and it can make two arms with near-identical per-test correctness look dramatically different.

Implication. Static API context costs little — no additional model calls beyond the first one-shot budget. It has a real, if not yet significant, effect on Java’s wrong-target problem, and a neutral effect on Python once measured fairly. Neither result changes the RQ1–RQ3 conclusions (Section IV): with or without context, gpt-4o-mini one-shot generation stays well short of the 80%/60% thresholds.

G. RQ5 (exploratory, post-hoc): one feedback/repair iteration on top of RQ4

We committed to a single repair round before running it, and ran it exactly once — iterating a technique until some threshold clears is precisely the drift our protocol rules out. Every RQ4 suite that was not yet effective went back to gpt-4o-mini with a real failure signal and a request for one corrective rewrite. For Java the signal was the 0%-coverage outcome plus the target’s API skeleton, since Maven/JVM error logs proved too noisy to serve as a clean compiler-diagnostic signal; for Python it was the actual fresh `pytest` failure output. Suites already effective at RQ4 (11/60 Java, matching Python’s per-test baseline) were carried forward unchanged rather than regenerated.

Java: further, still-inconclusive improvement. The effective rate climbs again, from 11/60 (18.3%, RQ4) to **15/60 (25.0%)**. Among the functions that changed, 4 improved and *none regressed* — repair matched or beat context on every paired function. The one-sided paired Wilcoxon test gives $p = 0.063$, still short of $\alpha = 0.05$ at this sample size, with a rank-biserial effect size of $r = 1.00$ (every non-tied pair favours repair) and a monotonic climb across all three stages (8.3% $\rightarrow$ 18.3% $\rightarrow$ 25.0%). Mutation score stays flat at a median of 0% ($n = 45$ paired functions), exactly as it was at RQ4. Repair gets GPT-4o-mini to hit the right lines more often; just as with context alone, the assertions still are not strong enough to kill mutants.

Python: a genuine regression, not a measurement artefact this time. Per-test-case pass rate *falls* to 15.5% (106/682 tests), below both RQ4 (33.3%) and the first no-context arm (32.2%). File count and suite length do not explain this one, unlike the RQ4-vs-original gap. Test volume grew again — 682 tests across 60 files, from 267 at RQ4 — so gpt-4o-mini appears to respond to a visible failure by *expanding* the suite rather than narrowly fixing the assertion that failed, and the added cases fail at a higher rate than the first ones survived at. We do not have a definitive mechanism, and the run-once commitment rules out chasing one post hoc. Whether the more-tests-not-better-tests response is a property of gpt-4o-mini specifically, of single-shot repair in general, or of the particular failure signals we fed it, remains open.

Overall verdict on RQ4+RQ5. Two rounds of low-cost, non-fine-tuning intervention leave Java with a real, if not significance-clearing, upward trend in coverage effectiveness and no downside anywhere, and Python with no benefit from context plus an outright regression from single-shot repair. Neither technique closes the gap to the pre-registered 80%/60% thresholds, alone or combined (Section IV): across the full $N = 120$ sample, the median stays at 0% for both metrics at every stage we tried. The interventions the literature recommends as next steps after one-shot prompting (Section II) are, on this evidence, no guaranteed fix: they help unevenly across languages, and in at least one case here (Python repair) they make things worse. “Add more scaffolding” does not straightforwardly improve LLM test generation. We agreed in advance not to keep searching for a technique that crosses the threshold, so we stop here rather than attempt a third intervention.

VI. THREATS TO VALIDITY

A. Construct Validity

Three times during this study a measurement produced a wrong result while the tooling signalled success, and each time only a cross-check against the artefacts on disk caught it. The first: mutation scores of 100% for test suites that had in fact *failed on the unmutated code* — every mutant was trivially “killed” because the suite was already broken. A green-on-original check now runs before any mutation analysis; a test that fails on the baseline source is scored INVALID outright. The second: EvoSuite returns exit code 0

even when it generates no tests whatsoever, and a run of 12 functions was initially logged as a complete success with zero test files on disk. The pipeline now checks for the generated file itself rather than trusting the exit code. The third: JaCoCo silently reported 0% coverage for EvoSuite tests, because EvoSuite’s separate instrumenting class loader changes class identity underneath it; disabling `separateClassLoader` during measurement fixed the attribution.

A fourth construct threat is disclosed rather than fixed: the two Python arms are scored by different mutation instruments — `mutmut` for the `gpt-4o-mini` suites, our custom AST engine for the Pynguin suites (Section III). Both enforce the same green-on-original gate and the same function-range scoping, but their operator sets differ, so the Python RQ2-B pairing compares scores produced by two different instruments. Given that Pynguin’s median coverage on this dataset was 0.0%, the direction of that comparison does not hinge on the instrument choice. Our own data also show why coverage alone is an unreliable proxy for test quality: one baseline suite reached 60% branch coverage on a function while killing 0% of its mutants, which is exactly why we report the coverage–mutation pair rather than either alone.

B. Internal Validity

The pilot run existed to surface integration defects before the main study, and it did. The first version of the Java dataset contained function files whose contents did not match their metadata — JA-002’s file held an unrelated `concat` method instead of `getOptionValues` — so the LLM was shown source code unrelated to the specified method. Every pilot LLM test was discarded and regenerated once the dataset was corrected. On the Python side, 10 out of 12 pilot measurements came back `INVALID` because functions had been extracted without their module context, an error unrelated to the LLM; measurement moved into real installed modules at pinned commits as a result. Baseline generation had its own defects: early experiments gave EvoSuite a 60-second search budget against Randoop’s 10 seconds, and Randoop’s output files were overwritten across runs. Both were fixed before the main experiments — equal budgets, per-function output names. Several tool–environment mismatches also had to be resolved: EvoSuite 1.2.0 ran under JDK 11 for most classes but needed a Java 8 runtime for the `csv/gson/joda` classes, so the generated suites mix JDK 11 (39 functions) and JDK 8 (21 functions) provenance; multi-release dependency JARs carried Java 21 bytecode that older ASM versions reject (`Unsupported class file major version 65`); and `commons-math` disables JaCoCo in its own pom via `jacoco.skip=true`. Four out of six measurement-pipeline failures terminated with exit code 0. Silent failures of that kind survive anything short of checking that the result artefacts actually exist.

C. External Validity

Every result here characterises the lightweight `gpt-4o-mini-2024-07-18` and only that model; extrapolating

to larger models such as GPT-4o is unwarranted (amendment v1.1). The dataset spans two ecosystems — Apache-style Java libraries and Python web libraries (`flask`, `requests`) — and is unevenly distributed within Python: 48 of 60 functions come from `flask`. Functions that depend on heavyweight environmental context, such as GUI or network access, are under-represented. Baseline coverage also varies sharply by repository, running near-perfect on numeric utility code but substantially lower on parser- and formatter-style classes, so a threshold like RQ1’s 80% target is repository-relative rather than a universal bar.

D. Conclusion Validity

With $N = 120$ functions (60 per language), we rely throughout on non-parametric tests — one-sample and paired Wilcoxon, Spearman correlation, $\alpha = 0.05$ — and report effect sizes alongside every p -value. The thresholds (80% coverage, 60% mutation) were fixed in the approved proposal before any full-run data existed, and every protocol change made along the way (model, dataset, prompting mode, measurement context) was recorded as an amendment before the corresponding data were produced — that ordering is what mitigates HARKing here. Mutation scores for 3/60 baseline cells — JA-017 (`jfreechart`), JA-048 and JA-058 (both `jsoup`) — were unavailable: PIT generated no mutants within these functions’ target line ranges under either baseline tool, so there was nothing to score. Branch coverage for these 3 functions is unaffected, since it is measured independently and before the PIT step, and remains in `metrics_full.csv`; only the mutation-score cell is reported as missing rather than imputed as zero, so the baseline’s RQ2 numbers are not silently deflated. The same three functions also appear among the LLM’s own `INVALID/non-effective` Java results (Section IV), for an entirely unrelated reason: wrong-target invocation. PIT’s missing mutants and the LLM’s wrong-target problem were diagnosed separately; the overlap is coincidence.

A separate environment issue hit only the Pynguin baseline (Python). Under Python 3.14, the interpreter used everywhere else in the measurement pipeline, Pynguin’s bytecode instrumentation produced empty test suites (0.0% coverage) for every module. Reproducing the failure with verbose logging traced it to Pynguin’s internal master–worker subprocess — a genuine tool–interpreter incompatibility rather than a configuration mistake. The fix was a dedicated Python 3.10 virtual environment for Pynguin invocations only, the interpreter version most Pynguin releases are actually validated against. It changes none of the pre-registered metric, threshold, dataset, or model, and it was applied uniformly before a single Pynguin measurement was taken, so it carries no HARKing risk. Even with the fix in place, Pynguin’s own median branch coverage on this dataset stayed at 0.0% (Section IV), which we read as a genuine, if weak, finding about Pynguin’s real-world difficulty on these modules rather than a leftover environment defect.

VII. CONCLUSION

This study measured GPT-4o-mini as a one-shot unit-test generator on 120 real-world Java and Python functions of medium cyclomatic complexity (CC 5–10), scoring every suite inside its host project and comparing against EvoSuite and Randoop (Java, equal 60-second budgets) and Pynguin (Python, 90-second budget).

RQ1 (coverage threshold). GPT-4o-mini does *not* achieve a median branch coverage $\geq 80\%$ at any level of analysis. Across all 120 sampled functions the median is 0% (H_0 not rejected, one-sample Wilcoxon). Even restricted to the 28 functions whose suites both compiled *and* exercised the target, the median stops at 75.0% ($p = 0.84$, $r = -0.22$). The bar itself is demanding: on the same 60 Java functions EvoSuite manages a median branch coverage of 55.0% and Randoop 21.1%, so no tool clears 80% on this CC 5–10 sample.

RQ2 (quality vs. baselines). Mutation scores are weaker still: over the suites with a measurable mutation score, the median is 36.84% for Python ($n = 17$) and 0% for Java ($n = 54$), both well under the 60% threshold. Paired Wilcoxon tests hand both Java baselines a decisive win — EvoSuite at $p < 0.001$, Randoop at $p = 0.003$, each with $r = -1.00$: not one function where the two suites differed went to GPT-4o-mini. Against Pynguin the comparison flips ($p = 0.010$, $r = +0.85$ in GPT-4o-mini’s favour), but Pynguin recorded just 0.0% median coverage on this dataset — a tooling failure examined in Section VI — so the win says nothing about the robustness of GPT-4o-mini’s own tests.

RQ3 (complexity correlation). Cyclomatic complexity shows no significant correlation with coverage inside the CC 5–10 band in either language (Python $\rho = +0.18$, $p = 0.41$; Java $\rho = +0.13$, $p = 0.83$; pooled $\rho = +0.16$, $p = 0.40$). Within this band, test effectiveness hinges on a binary outcome — whether the generated suite compiles and invokes the correct target (Section IV) — rather than on any continuous relationship with complexity. Locating the true “breaking point” would take a sample that extends below CC 5 and above CC 10.

Two lessons stand out. First, evaluating LLM-generated tests on functions extracted out of context understates the difficulty of real-world code: functions from mature projects lean on their surrounding module, and an evaluation that ignores that context flatters the model. Second, the measurement pipeline deserves the same scrutiny as the model. Most of the failures we hit were silent — exit code 0, no artefact — so every step had to be checked against a report file that actually exists on disk (Section VI). Coverage alone is likewise too coarse a signal; without the paired mutation analysis, apparently adequate suites would have been overrated.

Future work. RQ4 and RQ5, two exploratory post-hoc follow-ups (Sections V-F–V-G), asked whether low-cost prompting interventions could close the gap to the pre-registered thresholds. They could not. Java’s effective-suite rate did climb across the three stages — 8.3% $\rightarrow$ 18.3% $\rightarrow$ 25.0% with no intervention, real API context, and one

feedback/repair iteration respectively — one-sided $p = 0.074$ for the context step and $p = 0.063$ for the repair step, both short of significance at $n = 60$, while mutation score stayed at 0% throughout. Python gained nothing from either the added context or the feedback/repair step; the single-shot repair in fact produced a clear decline, most likely because gpt-4o-mini answered failures by padding the suite with more tests rather than by fixing the failed assertions. We stopped the exploratory arm at these two interventions and did not keep hunting for a technique that would cross the preset thresholds (anti-HARKing).

Five concrete directions remain: (1) expanding the Java sample to test whether the positive coverage trend from RQ4/RQ5 reaches statistical significance; (2) evaluating multi-iteration repair loops with explicit stopping criteria based on suite size, to determine whether Python’s regression is a property of single-shot repair or persists under further refinement; (3) replicating the analyses with larger LLMs (GPT-4o or Claude-class models) to isolate the role of model capacity; (4) broadening the dataset to additional Python libraries and to functions of higher cyclomatic complexity (CC > 10) to locate potential breaking points more precisely; and (5) integrating the generation–measurement pipeline into continuous integration workflows for ongoing, in situ assessment. All five call for pre-registered protocols.

On this evidence, one-shot LLM test generation is not yet a substitute for EvoSuite or Randoop on real-world code. Closing that gap will take feedback, retrieval, and repair mechanisms on the generation side, and context-aware, artefact-verified benchmarks on the evaluation side.

AI USE DISCLOSURE

This document was originally drafted by human authors. Automated evaluation and reporting sections, including test generation and analysis, were produced and refined using AI-based large language models.

REFERENCES

- [1] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, L. Zhang, A. R. Chen, and L. Ma, “TestEval: Benchmarking large language models for test case generation,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3547–3562.
- [2] A. Kumar *et al.*, “Empirical evaluation of advanced LLMs on automated unit test generation for Java applications,” *IEEE Access*, vol. 13, pp. 1245–1260, 2025.
- [3] Lira *et al.*, “Evaluating the effectiveness and cost-efficiency of large language models in automated unit test generation,” in *Anais do Simpósio Brasileiro de Qualidade de Software (SBQS)*, 2025. [Online]. Available: <https://sol.sbc.org.br/index.php/sbqs/article/view/39000>
- [4] A. Chudic and G. Çaliklı, “Automated test suite enhancement using large language models with few-shot prompting,” 2026.
- [5] K. Jasti, T. P. Sahu, R. Pentyala, M. Mohit, and V. Yelleti, “FeedbackLLM: Metadata-driven multi-agentic language-agnostic test case generator with evolving prompt and coverage feedback,” 2026.
- [6] M. Haratian *et al.*, “PR-aware test case generation: An empirical comparison of search-based and LLM-based approaches,” 2026.
- [7] G. Sapozhnikov *et al.*, “An empirical study on the structural coverage and compilation rate of ChatGPT-generated unit tests,” in *Proc. ICSE Companion*, 2024.
- [8] Q. Zhang, Y. Shang, C. Fang, S. Gu, J. Zhou, and Z. Chen, “TestBench: Evaluating class-level test case generation capability of large language models,” 2024.

- [9] D. Konstantinou *et al.*, “Evaluating plain-LLM test generation across diverse Java projects: A multi-model analysis,” 2026.
- [10] R. Jain and C. Le Goues, “TestForge: Feedback-driven, agentic test suite generation,” 2025.
- [11] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *Proc. ESEC/FSE*, 2011.
- [12] C. Pacheco and M. D. Ernst, “Randoop: Feedback-directed random testing for Java,” in *Proc. OOPSLA Companion*, 2007.
- [13] S. Lukaszcyk and G. Fraser, “Pynguin: Automated unit test generation for Python,” in *Proc. ICSE Companion*, 2022.
- [14] W. C. Ouédraogo *et al.*, “Prompt engineering in LLMs for automated unit test generation: A large-scale study,” *Empirical Software Engineering*, 2025.
- [15] S. Gu *et al.*, “Improving LLM-based unit test generation via template-based repair,” 2025, *aCM Trans. Softw. Eng. Methodol.* (TOSEM).
- [16] L. Broide, R. Stern, and A. Mordoch, “EvoGPT: Leveraging LLM-driven seed diversity to improve search-based test suite generation,” 2026.
- [17] Zeng *et al.*, “A logic-guided and explainable approach to LLM-based unit test generation,” *Applied Sciences*, vol. 16, no. 5, p. 2542, 2026.
- [18] M. Konstantinou, R. Degiovanni, J. M. Zhang, M. Harman, and M. Papadakis, “YATE: The role of test repair in LLM-based unit test generation,” 2025.
- [19] P. Xue, Y. Zhang, Z. Yang, X. Ren, X. Li, P. Hu, L. Wu, and K. Li, “DISTINCT: A description-guided branch-consistency analysis framework for non-regressive test case generation,” 2025, *IEEE Trans. Softw. Eng.* (TSE).
- [20] R. Liu *et al.*, “Type-aware LLM-based regression test generation for Python programs,” 2025.
- [21] G. Wang, C. Yang, X. Zhou, Z. Sun, B. Wang, D. Lo, and D. Hao, “TestDecision: Sequential test suite generation via greedy optimization and reinforcement learning,” 2026.
- [22] A. Moradi Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, “Effective test generation using pre-trained large language models and mutation testing,” 2023.
- [23] G. Antal *et al.*, “Leveraging GPT-4 for vulnerability-witnessing unit test generation,” in *Proc. EASE*, 2025.
- [24] Silva *et al.*, “LLMs as test generators: A comparative benchmarking study,” in *Anais do Simpósio Brasileiro de Engenharia de Software (SBES)*, 2025. [Online]. Available: <https://sol.sbc.org.br/index.php/sbes/article/view/36983>
- [25] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, “PIT: A practical mutation testing tool for Java,” in *Proc. ISSTA*, 2016.